

# INTEGRATED FULL-STACK AND AGENTIC AI APPLICATION DEVELOPMENT

## ASSIGNMENT 1 • GROUP ASSIGNMENT SPECIFICATION

|                        |                                                                                                                                                                                                                                 |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Programme              | BSc (Hons) in Information Technology, specializing in SE/AI                                                                                                                                                                     |
| Academic Level         | Year 3, Semester 1                                                                                                                                                                                                              |
| Assignment Type        | Group Full-Stack and Agentic AI Application Development Project                                                                                                                                                                 |
| Weighting              | <b>25% of the final module mark</b> (marked out of 100 and scaled). Together with the Mini Hackathon (15%), this forms the 40% Assignments component of SE3090.                                                                 |
| Group Size             | Normally 4 students; variations require written approval from the lecturer-in-charge.                                                                                                                                           |
| Duration               | 9 weeks (31 July – 30 September 2026)                                                                                                                                                                                           |
| Mandatory Technologies | ASP.NET Core Web API, PostgreSQL, React, Flutter and Agentic AI                                                                                                                                                                 |
| AI Use Level           | <b>Level 4 — Full AI.</b> Development AI use is allowed with disclosure. No external AI tools may be used during the final demonstration or viva; the submitted application's Agentic AI subsystem must be run. See Section 18. |
| Evaluation             | One final evaluation: demonstration + viva, 100 marks (30 group / 70 individual)                                                                                                                                                |
| Release Date           | <b>Friday, 31 July 2026</b> — published through the official Learning Management System (Course Web)                                                                                                                            |
| Due Date               | <b>Wednesday, 30 September 2026 at 11:50 PM</b> — one submission by the nominated group leader through Course Web                                                                                                               |

**Important:** Students must read the complete specification before selecting a domain or beginning implementation. Working software, traceable individual contribution, tests, documentation, deployment evidence and viva understanding are all required.

## 1. Assignment Overview

Each group must design, implement, integrate, test and deploy one coherent software system that combines a web application, mobile application, RESTful backend, relational database and a meaningful Agentic AI workflow. The system must solve a credible real-world problem and demonstrate how modern software engineering frameworks work together.

**Integrated-system rule:** The React and Flutter applications must use the same ASP.NET Core Web API, PostgreSQL database, user identity, permissions and business rules. Disconnected prototypes will not satisfy the assignment.

### 1.1 Assignment Objectives

- Apply ASP.NET Core, PostgreSQL, React and Flutter to a realistic full-stack problem.
- Design secure REST APIs, relational data models, web interfaces and mobile workflows.
- Implement a controlled Agentic AI workflow that plans, delegates, uses tools, validates results and requests human approval when required.
- Use Git, GitHub, automated testing, code review, CI/CD, documentation and deployment as part of professional software development.
- Justify framework and architecture decisions in writing through an Architecture Decision Record (ADR).
- Demonstrate individual technical ownership and the ability to explain, modify and debug submitted work.

### 1.2 Learning Outcome Alignment

This assignment assesses all four learning outcomes of SE3090, as stated in the approved module outline:

| Learning Outcome | Official Module Outline Wording                                                                                                                             |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| LO1              | Evaluate different types of software engineering frameworks used in web, mobile, cloud, and AI-assisted software development.                               |
| LO2              | Apply suitable frameworks and tools to build web, mobile, and full-stack software applications efficiently and effectively.                                 |
| LO3              | Use best practices for integrating frameworks, managing collaborative development, applying CI/CD, ensuring code quality, and deploying software solutions. |
| LO4              | Select appropriate frameworks, tools, and agentic AI-assisted approaches to meet specific project and industry requirements.                                |

## 2. Required Technology Stack

| Area    | Requirement                                                        |
|---------|--------------------------------------------------------------------|
| Backend | C# and ASP.NET Core Web API. This is the mandatory public backend. |

| Area               | Requirement                                                                                                                                                                     |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Data Access        | Entity Framework Core with the PostgreSQL provider.                                                                                                                             |
| Database           | PostgreSQL.                                                                                                                                                                     |
| Web Application    | React using functional components, hooks, routing and a justified state-management approach.                                                                                    |
| Mobile Application | Flutter and Dart using a justified state-management approach.                                                                                                                   |
| Agentic AI         | Any suitable and justified framework, such as LangGraph (the stack used in labs), Microsoft Agent Framework, LlamaIndex agents, Google ADK, or a custom orchestration approach. |
| Version Control    | Git and GitHub from the beginning of the project, including a GitHub Actions CI workflow (see Section 13).                                                                      |
| Testing            | Suitable tools for backend, React, Flutter, integration, performance and Agentic AI evaluation.                                                                                 |

**Mandatory backend rule:** React and Flutter must communicate only with the ASP.NET Core Web API. Where a Python Agentic AI service is used, it must operate as an internal service called by ASP.NET Core and must not be called directly by either client application.

### 3. Group Structure and Individual Contribution

The standard group size is four students. Each student must take primary ownership of one business component, so a four-student group must implement four primary components. If the lecturer-in-charge approves a different group size, the number of primary components must equal the number of students (for example, five students = five components). The lecturer-in-charge will confirm any proportional changes to the Agentic AI contributions and overall functional scope in writing.

| Student   | Primary Component | Required Individual Evidence                                                                                  |
|-----------|-------------------|---------------------------------------------------------------------------------------------------------------|
| Student 1 | Component A       | Backend, database, React, Flutter, tests, Git evidence, documentation and a distinct Agentic AI contribution. |
| Student 2 | Component B       | Backend, database, React, Flutter, tests, Git evidence, documentation and a distinct Agentic AI contribution. |
| Student 3 | Component C       | Backend, database, React, Flutter, tests, Git evidence, documentation and a distinct Agentic AI contribution. |
| Student 4 | Component D       | Backend, database, React, Flutter, tests, Git evidence, documentation and a distinct Agentic AI contribution. |

- There must be no project-manager-only, testing-only or documentation-only roles.

- Every student must contribute technically across the required stack and must have an identifiable Agentic AI contribution.
- Individual marks may be adjusted using Git history, pull requests, issue ownership, test evidence, code ownership, and responses to embedded viva and technical questions.
- Code or features that a student cannot explain, modify or debug may receive reduced or zero individual marks.

## 4. Domain and Functional Scope

Each group must select a unique real-world domain. Suggested domains include healthcare appointments, event management, travel planning, inventory, vehicle services, education, property rental, food delivery, recruitment, agriculture, tourism, help-desk systems and community services.

### 4.1 Minimum Domain Complexity

- At least three user roles with different responsibilities and permissions.
- For the standard four-student group, include at least four major business components with relational data and business-specific operations. An approved group-size variation must follow the one-component-per-student rule in Section 3.
- CRUD operations plus status workflows, search, filtering, sorting, pagination and reporting or analytics.
- Meaningful and different purposes for the React and Flutter applications.
- At least one third-party service integration.
- At least one complete cross-platform workflow involving React, Flutter, ASP.NET Core, PostgreSQL and Agentic AI.

## 5. Part 1 – Secure ASP.NET Core RESTful API Backend

The ASP.NET Core backend is the authoritative application layer for public REST APIs, authentication, authorization, validation, business rules, persistence, Agentic AI workflow initiation, approval and audit logging.

| Area              | Minimum Requirement                                                                                           |
|-------------------|---------------------------------------------------------------------------------------------------------------|
| Architecture      | Controllers, DTOs, service/application layer, suitable data-access abstraction and dependency injection.      |
| REST API          | Correct routes, HTTP methods, status codes, request/response models and asynchronous operations.              |
| Security          | JWT authentication, role-based authorization, protected endpoints, password hashing and secure configuration. |
| Data Operations   | CRUD, search, filtering, sorting, pagination, history and business-specific operations.                       |
| Quality           | Server-side validation, global error handling, structured logging, CORS and Swagger/OpenAPI.                  |
| Agent Integration | Endpoints for starting workflows, reviewing status, human approval and viewing execution summaries.           |

**Individual component minimum:** Each student-owned component must include at least four meaningful API endpoints and at least one business-specific operation beyond basic CRUD.

## 6. Part 2 – PostgreSQL Database

- Design a normalized relational database with an ER diagram and clear relational schema.
- Use primary keys, foreign keys, appropriate relationships, constraints, indexes and suitable PostgreSQL data types.
- Use Entity Framework Core migrations and suitable seed data.
- Apply transactions where required and maintain audit fields such as CreatedAt and UpdatedAt.
- Persist only the Agentic AI workflow state and execution summaries required by the design; do not store hidden reasoning, passwords, tokens or unnecessary sensitive data.

## 7. Part 3 – React Web Application

The React application should primarily support administrative, staff, dashboard, reporting, business-data management, Agentic AI monitoring and approval functions.

- Functional components, React Hooks, React Router and reusable component design.
- A suitable state-management approach such as Context API, Redux Toolkit, Zustand or another justified option.
- Complete ASP.NET Core API integration with protected routes and role-based navigation.
- CRUD interfaces, validation, search, filters, sorting, pagination and dashboard views.
- Responsive and accessible UI with loading, empty, success and error states.
- Agent workflow monitoring, execution summaries and approve/reject/revise controls where relevant.

## 8. Part 4 – Flutter Mobile Application

The Flutter application should primarily support user-facing or operational workflows. It must be a genuine mobile application that consumes the shared ASP.NET Core API.

- Reusable widgets, navigation/routing and a suitable state-management approach.
- Registration, login, logout, secure token storage and protected screens.
- Forms, validation, search, filtering, main business transactions, status tracking and history.
- Responsive layouts with loading, empty and error states.
- Agentic task submission, recommendation display and workflow status where suitable.
- At least one meaningful device feature, such as camera/image picker, GPS/map, QR scanning, file upload, notifications or date/time selection.

## 9. Part 5 – Agentic AI Subsystem

The Agentic AI feature must solve a meaningful, domain-relevant, multi-step problem. It must not be limited to a generic chatbot, FAQ interface, single-prompt workflow or simple text generator.

**Minimum acceptance rule:** The group must demonstrate at least one complete assessed workflow that satisfies every step in the “Minimum assessed workflow” row below.

## 9.1 Minimum Agentic AI Requirements

| Requirement                             | Expected Behaviour                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Minimum assessed workflow</b>        | At least one assessed workflow must receive a domain objective; create a structured multi-step plan; delegate steps to distinct agent roles; call allow-listed tools using validated inputs and structured outputs; persist workflow state; apply deterministic checks such as schema or business-rule validation; pause a defined high-impact action for approval by an authorized user; and produce either an auditable result or a safe, clearly recorded failure. |
| <b>What counts as a distinct agent?</b> | An agent counts as distinct only when it has an identifiable responsibility, a defined input and output contract, controlled tool permissions and visible participation in the workflow. Renaming the same prompt or copying identical behaviour does not count as a separate agent.                                                                                                                                                                                  |
| <b>Specialized agents</b>               | For the standard group, implement at least <b>four distinct agents</b> with clearly different responsibilities, such as planning or coordination, domain analysis, action or tool use, and validation or safety. Any approved adjustment must be confirmed in writing by the lecturer-in-charge.                                                                                                                                                                      |
| <b>Planning and delegation</b>          | The system analyses a user objective, creates a structured multi-step plan and delegates each step to an appropriate agent.                                                                                                                                                                                                                                                                                                                                           |
| <b>Controlled tools</b>                 | Agents may use only allow-listed tools. Validate every tool input, return structured outputs, handle errors and apply least-privilege access.                                                                                                                                                                                                                                                                                                                         |
| <b>Shared state</b>                     | Persist the workflow ID, objective, plan, completed steps, tool results, validation results, errors, approval status and final outcome in structured, durable storage.                                                                                                                                                                                                                                                                                                |
| <b>Validation</b>                       | Apply deterministic validation, such as schema checks and business rules, before accepting outputs or allowing high-impact actions. Unsupported or unsafe actions must be rejected or returned for revision.                                                                                                                                                                                                                                                          |
| <b>Human approval</b>                   | At least one clearly defined high-impact action must pause until an authorized user approves, rejects or requests revision.                                                                                                                                                                                                                                                                                                                                           |
| <b>Observability</b>                    | Store or display auditable execution summaries, tool calls, timings, validation results, errors, retries, approval decisions and the final result or safe-failure outcome.                                                                                                                                                                                                                                                                                            |
| <b>Security</b>                         | Apply role-based access, prompt and tool-input validation, output validation, secret protection, timeouts, retry limits and safe failure behaviour.                                                                                                                                                                                                                                                                                                                   |

**Implementation flexibility:** Students may use any suitable Agentic AI framework, model and orchestration method. The selected approach must meet the minimum acceptance scenario, be justified in the ADR, run reliably during evaluation, be secured and be integrated through ASP.NET Core.

## 10. Required Integrated Architecture

### SE3090 Reference Integration Architecture

A generic required structure; groups must design and justify their own domain architecture.

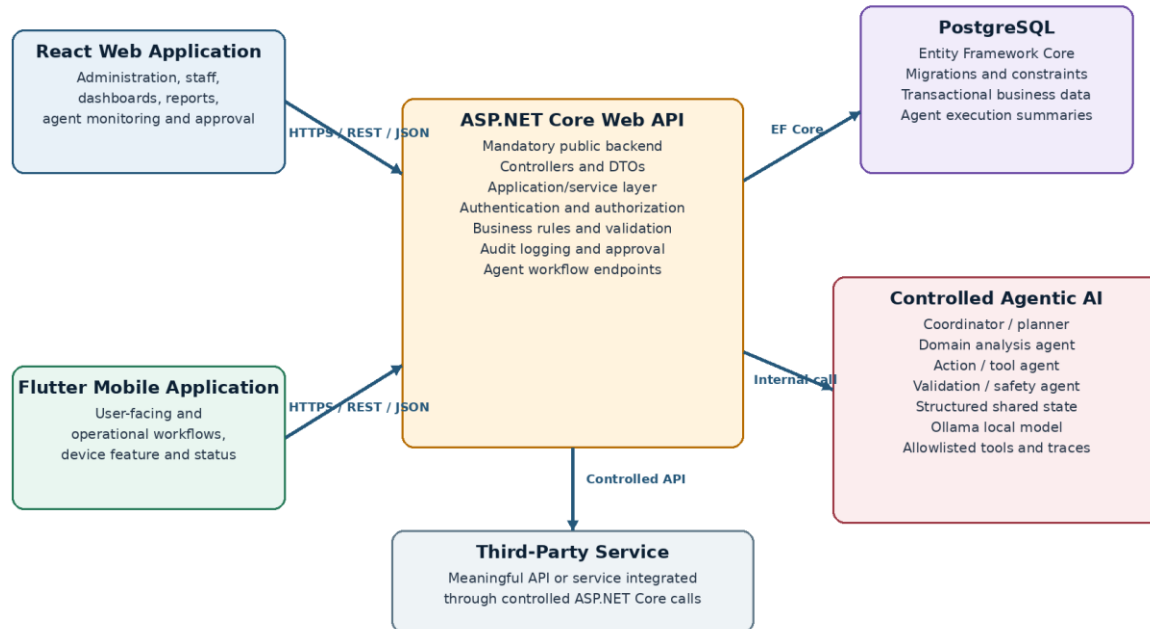
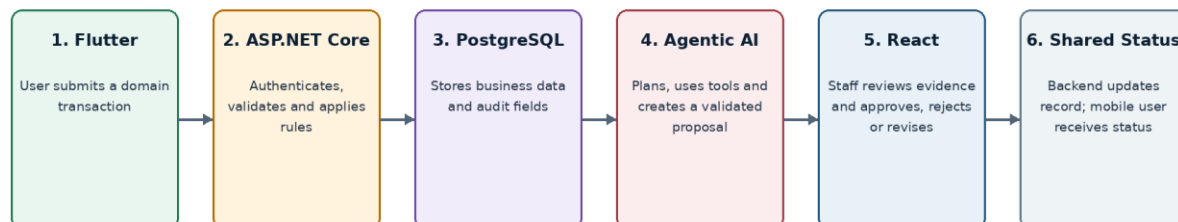


Figure 1. Reference integration architecture. Groups must adapt and justify the architecture for their selected domain.

### Required Cross-Platform Workflow Pattern

At least one workflow must begin in one client and continue through the other client.



Evidence must show consistent identities, permissions, business rules, data and status across both applications.

Figure 2. Required cross-platform workflow pattern.

**End-to-end evidence:** At least one demonstrated workflow must begin in one client, pass through ASP.NET Core, PostgreSQL and Agentic AI, require review or approval in the other client, and return an updated status to the initiating user.



## 11. Third-Party Integration

Each system must integrate at least one meaningful third-party API or service, such as maps, weather, currency, email/SMS, payment sandbox, calendar, cloud storage, QR service or notifications.

- Explain the business purpose and user benefit.
- Route external-service access through the ASP.NET Core backend where appropriate.
- Protect credentials and environment variables.
- Handle timeouts, invalid responses, service failures and rate limits.
- Validate and minimize any personal or sensitive data shared with the service.

## 12. Testing Requirements

| Area                                                                                                                                                                                                                                                     | Required Evidence                                                                                                                                                                                                                                                                                                                 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Backend                                                                                                                                                                                                                                                  | Unit, service-layer, validation, authentication/authorization, controller and API integration tests.                                                                                                                                                                                                                              |
| Database                                                                                                                                                                                                                                                 | PostgreSQL integration tests, constraints, migrations and transaction behaviour.                                                                                                                                                                                                                                                  |
| React                                                                                                                                                                                                                                                    | Component, form-validation, protected-route, API-integration and error-state tests.                                                                                                                                                                                                                                               |
| Flutter                                                                                                                                                                                                                                                  | Unit, widget, form-validation, navigation and API-integration tests.                                                                                                                                                                                                                                                              |
| End to End                                                                                                                                                                                                                                               | At least one complete Flutter/React – ASP.NET Core – PostgreSQL – Agentic AI workflow.                                                                                                                                                                                                                                            |
| Performance                                                                                                                                                                                                                                              | Concurrent requests, response time, success/failure rate, database response and Agentic AI latency.                                                                                                                                                                                                                               |
| Agent Evaluation                                                                                                                                                                                                                                         | Evidence that at least one complete minimum acceptance workflow passes a suitable golden case, including correct planning and delegation, agent and tool selection, structured outputs, deterministic validation, business-rule compliance, approval enforcement, prompt-injection resistance, failure recovery and safe failure. |
| <b>Agent evaluation rule:</b> LLM-as-a-judge may be used as supporting evidence, but it must not be the only evaluation method. Use rule-based assertions, schema validation, golden cases, deterministic validators and human review where appropriate. |                                                                                                                                                                                                                                                                                                                                   |

## 13. Git, CI/CD and Collaborative Development

- Create the GitHub repository at the beginning of the project.
- Use meaningful commits, feature branches, issues, pull requests, reviews and a project board.
- Configure at least one **GitHub Actions CI workflow** that restores, builds and runs the automated backend tests on every push and pull request to the main branch. Additional pipelines (frontend build, lint, Flutter analyze, deployment) are encouraged.
- Maintain clear evidence of task allocation, merge management and conflict resolution.
- Each student must show regular technical contribution across the project lifecycle.
- Artificial commit activity, final-day bulk uploads or unexplained copied code will not be accepted as evidence of contribution.



## 14. Deployment and Documentation

| Component        | Deployment Requirement                                                                                        |
|------------------|---------------------------------------------------------------------------------------------------------------|
| ASP.NET Core API | Deploy to a suitable cloud platform and provide a working health URL and Swagger URL.                         |
| PostgreSQL       | Deploy securely with migrations, restricted credentials and initialization instructions.                      |
| React            | Deploy and provide a working live URL configured to use the deployed API.                                     |
| Flutter          | Submit complete source code and a runnable Android APK or approved equivalent.                                |
| Agentic AI       | Deploy or run locally as appropriate; provide complete setup, model/framework requirements and startup order. |

**Service cost and availability:** You must be able to complete this assignment using institution-provided or no-cost services. Paid subscriptions are not required. Keep clear local setup instructions. If a required external service has a confirmed outage near submission or evaluation, inform that to evaluator and provide evidence of the outage. When you doing the evolution.

### 14.1 README and Technical Documentation

- Project overview, business problem, user roles, features and technology justification.
- System architecture, Agentic AI architecture, database design and repository structure.
- Installation, environment variables, database setup and startup instructions for all components.
- API documentation, test instructions, deployment instructions, live URLs and test accounts.
- Individual contributions, challenges, security considerations and AI usage declaration.

### 14.2 Architecture Decision Record (ADR)

As introduced in Lecture 01, each group must submit an **Architecture Decision Record** — a short document (one page per decision) capturing the context, options considered, decision and consequences for the group's key technical choices. The ADR is the primary written evidence for LO4 and will be referenced during the viva.

At minimum, record decisions for: the state-management approach in React and in Flutter, the Agentic AI framework and orchestration method, the database schema strategy for agent workflow state, and the cloud deployment platform. Three to six decisions is a typical, healthy range.

## 15. Submission Guidelines

| Submission Item                       | What You Must Submit                                                                                                                                                                                                                                                                                                                                                                       |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Group leader and deadline</b>      | Group leader must make the only group submission through Course Web by 11:50 PM on Wednesday, 30 September 2026.                                                                                                                                                                                                                                                                           |
| <b>One consolidated report (PDF)</b>  | Combine all written work into one clearly organized PDF. Do not upload the group and individual written reports as separate files.                                                                                                                                                                                                                                                         |
| <b>Group Report section</b>           | Include the project overview and scope; requirements and user roles; full-stack and Agentic AI architecture; database and ER diagram; API, React and Flutter design; technical report; software testing report; Agentic AI evaluation report; performance report; deployment report; ADRs; security considerations; diagrams; references; and the consolidated group AI usage declaration. |
| <b>Individual Report sections</b>     | Include one clearly labelled section for each student containing the contribution statement; owned component and technical work; key commit, pull-request and test evidence; challenges and learning; individual AI usage log; approximately one-page AI reflection; and signed declaration.                                                                                               |
| <b>Repository and deployed system</b> | Include the repository URL, React URL, ASP.NET Core API or health URL, Swagger URL, PostgreSQL deployment evidence, Agentic AI setup or access information, required environment-variable names and startup instructions.                                                                                                                                                                  |
| <b>Flutter APK</b>                    | Submit a runnable Android APK, or another format approved in writing, together with installation instructions.                                                                                                                                                                                                                                                                             |
| <b>Demonstration video</b>            | Provide a working demonstration-video (10 min) link. Sharing must be set so that anyone with the link can view it without requesting access.                                                                                                                                                                                                                                               |
| <b>Required access period</b>         | Keep the repository, demonstration video and all deployed services accessible to evaluators until at least Wednesday, 21 October 2026 (three weeks after the submission deadline).                                                                                                                                                                                                         |

**Before submitting:** The group leader must open every submitted link in a private or incognito browser and confirm that evaluators can access it. Only one submission is required per group.

*Within the single consolidated report, the following section lengths are suggested for guidance only: technical report 10–15 pages; testing report 6–10 pages; Agentic AI evaluation report 5–8 pages; performance report 3–5 pages; deployment report 3–5 pages; and ADRs 3–6 pages. These are not graded limits; the quality and relevance of the evidence matter more than page count.*

**Naming convention:** Use **SE3090\_GroupNumber** for all submitted items. (e.g. SE3090\_G07)

## 16. Final Evaluation – Complete Integrated System

**Total: 100 marks** | Group contribution: 30 marks | Individual contribution: 70 marks. The mark out of 100 is scaled to **25% of the final module mark**.

The assignment will be assessed through one final evaluation only. The group must deliver a 10-minute demonstration of the complete integrated system, followed by a 20-minute viva and technical question session. Every student must be present and may be asked to explain, modify, test or debug their individual contribution.

| Contribution | Criterion                                                         | Marks |
|--------------|-------------------------------------------------------------------|-------|
| Group        | Component Design and Business Logic                               | 10    |
| Group        | Integrated Architecture, Agent Orchestration and State Management | 10    |
| Group        | Documentation and Deployment                                      | 10    |
| Individual   | ASP.NET Core RESTful API Development                              | 10    |
| Individual   | PostgreSQL Integration and Data Modelling                         | 10    |
| Individual   | React Web Application                                             | 10    |
| Individual   | Flutter Mobile Application                                        | 10    |
| Individual   | Individual Agentic AI Contribution                                | 12    |
| Individual   | API Integration, Security and Cross-Platform Functionality        | 10    |
| Individual   | Testing, CI and Git Workflow                                      | 8     |

**AI use during the evaluation:** During the final demonstration and viva, students may not use external AI assistants, chatbots, IDE copilots or agentic coding tools to answer questions, generate explanations or modify the submitted work. The Agentic AI subsystem implemented as part of the submitted application must be executed during the demonstration.

## 16.1 Marking Rubric – Final Evaluation

Complete Integrated Full-Stack and Agentic AI System | Total: 100 Marks

### Group Contribution (30 Marks)

| Criterion                                                                     | Excellent                                                                                                                                                                                                                                                                                   | Good                                                                                                                                                                               | Satisfactory                                                                                                                                                                                   | Poor                                                                                                                                                   | Very Poor                                                                                                                                                                                      |
|-------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Component Design and Business Logic (10)</b>                               | All major components are clearly defined and fully functional. Business rules are correctly implemented through suitable services and support a complete end-to-end workflow. — 10 marks                                                                                                    | Main components and business rules work with only minor functional or architectural issues. — 8 marks                                                                              | Core components and main business rules function, and a basic end-to-end workflow is demonstrated; some secondary rules, edge cases or integration remain incomplete. — 6 marks                | Some components or business rules are implemented, but workflows are fragmented, unreliable or substantially incomplete. — 4 marks                     | Components are poorly structured, mostly incomplete or fail to implement the stated business requirements. — 2 marks                                                                           |
| <b>Integrated Architecture, Agent Orchestration and State Management (10)</b> | Complete full-stack integration and one complete minimum Agentic AI acceptance workflow are demonstrated. Distinct agents, persisted state, allow-listed tools, deterministic validation, auditable logs, safe failure and authorized human approval all work correctly. — 10 marks         | The integrated workflow meets the minimum acceptance scenario, with only minor issues in orchestration, state, logging, tool controls, validation, approval or recovery. — 8 marks | The core integrated workflow works and most acceptance elements are present, but one or more elements are only partly effective or supported by limited evidence. — 6 marks                    | Only a partial integrated workflow works; several mandatory acceptance elements are missing, unreliable or weakly integrated. — 4 marks                | No complete assessed workflow is demonstrated; agents are not distinct, state, tools, validation or approval are absent, or the feature is only a chatbot or disconnected prototype. — 2 marks |
| <b>Documentation and Deployment (10)</b>                                      | The consolidated report is complete, well organized and contains all Group Report and Individual Report sections, ADRs, AI usage documents, evidence and working links. Required systems are deployed, the APK works, evaluator access is clear and setup is fully reproducible. — 10 marks | The consolidated report, access information and deployment are mostly complete, with only minor missing evidence, link or setup issues. — 8 marks                                  | The main Group Report and Individual Report sections and core deployment instructions are provided, but several evidence items, links, AI documents or setup details are incomplete. — 6 marks | The consolidated report is limited or poorly organized, and deployment or evaluator access is only partly working or difficult to reproduce. — 4 marks | The consolidated report, required group or individual sections, or access details are missing, and major components cannot be deployed or executed. — 2 marks                                  |

### Individual Contribution (70 Marks)

| Criterion                                        | Excellent                                                                                                                                                                                                    | Good                                                                                                                                          | Satisfactory                                                                                                                                                 | Poor                                                                                                                                                              | Very Poor                                                                                                |
|--------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <b>ASP.NET Core RESTful API Development (10)</b> | API component is complete and follows REST conventions with DTOs, validation, security, async operations, suitable architecture, exception handling and correct status codes. The student accurately answers | Main API functionality works with minor REST, validation, security or architecture issues. The student answers most related questions and can | Core CRUD and API operations work, but notable gaps remain. The student answers routine questions or makes a simple change but has technical gaps. — 6 marks | Only limited API functionality works; major areas are incomplete or unreliable. The student struggles to answer questions or modify and debug the work. — 4 marks | The API contribution is missing or non-functional, or the student cannot explain or modify it. — 2 marks |

| Criterion                                             | Excellent                                                                                                                                                                                                                                                                                                      | Good                                                                                                                                                                                                                                                                | Satisfactory                                                                                                                                                                                                                   | Poor                                                                                                                                                                                     | Very Poor                                                                                                                                     |
|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
|                                                       | related viva questions and can explain, test, modify or debug the contribution. — 10 marks                                                                                                                                                                                                                     | explain a suitable change. — 8 marks                                                                                                                                                                                                                                |                                                                                                                                                                                                                                |                                                                                                                                                                                          |                                                                                                                                               |
| <b>PostgreSQL Integration and Data Modelling (10)</b> | The contribution demonstrates suitable entities, relationships, constraints, normalization, migrations, indexing and data integrity. The student accurately answers related viva questions and can explain, trace, modify or debug the database contribution. — 10 marks                                       | The database is functional and suitably modelled, with minor design or integration issues. The student answers most questions and can explain or make a suitable database change. — 8 marks                                                                         | Basic database integration works, but notable gaps remain. The student answers routine questions but has difficulty explaining some relationships, constraints or migrations. — 6 marks                                        | Database integration is limited, incomplete or inconsistent. The student provides weak answers and struggles to modify or debug it. — 4 marks                                            | The database contribution is missing or non-functional, or the student cannot explain the schema and integration. — 2 marks                   |
| <b>React Web Application (10)</b>                     | Uses reusable components, routing, suitable state management, protected routes, validation, responsive UI, loading and error states, and complete API integration. The student accurately answers related viva questions and can explain, modify or debug the React contribution. — 10 marks                   | Main React functionality works with minor issues in structure, state, UI, validation or error handling. The student answers most questions and can complete and explain a suitable change. — 8 marks                                                                | Core React screens and API operations work, but notable gaps remain. The student answers routine questions or completes a simple change with some difficulty. — 6 marks                                                        | Limited React functionality is demonstrated. The student struggles to answer questions or modify and debug the application. — 4 marks                                                    | The React contribution is missing or non-functional, or the student cannot explain or modify it. — 2 marks                                    |
| <b>Flutter Mobile Application (10)</b>                | Contains reusable widgets, routing, state management, secure API integration, validation, responsive screens, loading and error states, and a meaningful device feature. The student accurately answers related viva questions and can explain, modify or debug the Flutter contribution. — 10 marks           | Main Flutter functionality works with minor issues in architecture, state, UI or API handling. The student answers most questions and can complete and explain a suitable change. — 8 marks                                                                         | Core Flutter screens and API communication work, but notable gaps remain. The student answers routine questions or completes a simple change with some difficulty. — 6 marks                                                   | Limited Flutter functionality is demonstrated. The student struggles to answer questions or modify and debug the application. — 4 marks                                                  | The Flutter contribution is missing or non-functional, or the student cannot explain or modify it. — 2 marks                                  |
| <b>Individual Agentic AI Contribution (12)</b>        | A distinct, domain-relevant Agentic AI contribution is functional and integrated, with an identifiable responsibility, defined input and output contract, controlled tool permissions, validation, error handling, security, documentation and tests. The student accurately explains the agent, tools, state, | The distinct Agentic AI contribution is functional and relevant, with only minor issues in its contract, permissions, validation, security, testing, observability or integration. The student answers most questions and can explain a suitable change. — 10 marks | A basic but identifiable Agentic AI contribution participates in the workflow, but its contract, controls, tests or integration are incomplete. The student answers routine questions but demonstrates notable gaps. — 7 marks | A limited Agentic AI prototype is shown, but its responsibility or participation is unclear. The student struggles to explain the agent's behaviour, tools, state or controls. — 5 marks | The contribution is missing, non-functional, disconnected or duplicates another agent, and the student cannot explain or modify it. — 2 marks |

| Criterion                                                              | Excellent                                                                                                                                                                                                                                                                                                   | Good                                                                                                                                                                                             | Satisfactory                                                                                                                                                                                             | Poor                                                                                                                                                  | Very Poor                                                                                                                     |
|------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
|                                                                        | validation and approval flow and can modify or debug the contribution. — 12 marks                                                                                                                                                                                                                           |                                                                                                                                                                                                  |                                                                                                                                                                                                          |                                                                                                                                                       |                                                                                                                               |
| <b>API Integration, Security and Cross-Platform Functionality (10)</b> | React and Flutter use the same API. Authentication, authorization, token handling, validation, shared data, Agentic AI approvals and security controls work correctly. The student accurately answers related viva questions and can trace, modify or debug the complete workflow. — 10 marks               | Most integration and security functions work with minor inconsistencies or incomplete edge cases. The student answers most questions and can explain the main cross-platform workflow. — 8 marks | The shared API and core cross-platform workflow function, but notable gaps remain. The student answers routine questions but has difficulty explaining some security or integration decisions. — 6 marks | Only limited integration is demonstrated. The student provides weak answers and struggles to trace or debug the workflow. — 4 marks                   | Integration is absent or non-functional, or the student cannot explain the shared workflow and security controls. — 2 marks   |
| <b>Testing, CI and Git Workflow (8)</b>                                | Comprehensive tests cover the required layers and Agentic AI. CI passes, and Git history shows regular reviewed contributions with clear ownership. The student accurately answers related viva questions, explains the tests, CI workflow and Git evidence, and can diagnose a relevant failure. — 8 marks | Suitable testing, CI and Git practices are demonstrated with only minor missing evidence. The student answers most related questions correctly. — 6 marks                                        | Some relevant tests and basic Git and CI evidence are provided. The student answers routine questions but demonstrates limited understanding of coverage or workflow decisions. — 4 marks                | Few meaningful tests are provided, CI is absent or unreliable, and Git evidence is weak. The student struggles to answer related questions. — 2 marks | Almost no meaningful testing, CI or Git evidence is provided, and the student cannot explain the available evidence. — 1 mark |

**Final Evaluation Total = 30 Group + 70 Individual = 100 Marks → scaled to 25% of the module mark.**

*Rubric application and viva note: The five listed performance levels represent performance-band anchors, and evaluators may award intermediate marks. **To receive full marks for an individual criterion, the student must demonstrate the required work, correctly answer the related viva questions and, where requested, explain, test, modify or debug the work. If the student cannot demonstrate understanding or ownership, the criterion mark will be reduced; where no relevant evidence is provided, zero marks may be awarded.***

## 17. Demonstration and Viva Requirements

### 17.1 Demonstration Checklist

- ☐ Login using different roles and demonstrate protected operations.
- ☐ Demonstrate CRUD and a business-specific workflow and show PostgreSQL data changes and Swagger documentation.
- ☐ Demonstrate React and Flutter using the same ASP.NET Core API.
- ☐ Run the application's Agentic AI subsystem and demonstrate the complete minimum acceptance workflow: domain objective, structured plan, distinct agent roles, allow-listed tool use, persisted state, deterministic validation, authorized approval, and an auditable result or safe failure.
- ☐ Demonstrate human approval and execution-history summaries.
- ☐ Show error handling, tests, the passing CI workflow, deployed applications and GitHub contribution history.

### 17.2 Viva Scope

- Explain a controller, service, DTO, database relationship, constraint, migration or index.
- Explain authentication, authorization, state management, secure storage and API integration.
- Explain an agent role, tool, orchestration decision, shared state, validation, security control and human approval.
- Explain a test, Git contribution, CI workflow step, deployment decision, third-party integration or a decision recorded in the ADR.
- Modify a small feature, validation rule or business rule, or debug a failed workflow.

## 18. Usage of AI

This assessment is designed using the CLEAR Framework and the AI Assessment Scale (Perkins, Furze, Roe & MacVaugh, 2024). The permitted level of AI use, the tasks it applies to, the disclosure required and the marks awarded for process are set out below.

**AI Use Level — Level 4 (Full AI):** AI tools may be used extensively during the permitted development tasks in Section 18.1 when all use is disclosed, verified and understood. During the final demonstration and viva, no external AI assistant, chatbot, IDE copilot or agentic coding tool may be used to answer questions, generate explanations or modify the submitted work (Level 1 — No AI).

### 18.1 Where AI Tools May Be Used

AI tools — chat assistants, IDE copilots and agentic coding tools — may be used for the tasks below. In every case the group remains fully responsible for the correctness, security, originality and understanding of what is submitted.

| Task / Section                                    | Permitted use of AI tools                                                                                                               |
|---------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Domain &amp; requirements</b><br>(Section 4)   | Brainstorm domains, roles, features and user stories; research the domain. The final scope must be your own and defensible at the viva. |
| <b>Architecture &amp; ADR</b> (Sections 10, 14.2) | Explore options, compare frameworks, draft the ADR. The decision taken and its justification must be the group's own reasoning.         |



| Task / Section                                            | Permitted use of AI tools                                                                                                                                                          |
|-----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Backend &amp; database</b> (Sections 5, 6)             | Generate, refactor and debug ASP.NET Core code; design schema, migrations, seed data, indexes and the ER diagram. All owner-reviewed and tested.                                   |
| <b>React &amp; Flutter</b> (Sections 7, 8)                | Scaffold components and widgets, state management, routing, validation, styling, secure storage and device features.                                                               |
| <b>Agentic AI subsystem</b> (Section 9)                   | Agent design, prompt engineering, tool definitions, orchestration, validation and safety controls. Prompt engineering and human-in-the-loop validation are examinable at the viva. |
| <b>Testing, CI/CD &amp; deployment</b> (Sections 12–14)   | Generate and run tests and test data; write CI workflows; configure and troubleshoot deployment. Tests must be executed and understood, not only generated.                        |
| <b>Reports, README &amp; diagrams</b> (Sections 14.1, 15) | Draft, structure and proofread. All facts, figures, screenshots, test results and evaluation findings must be your own.                                                            |

## 18.2 Where AI Tools May Not Be Used

- **Final demonstration and viva.** This evaluation is conducted under Level 1 (No AI). You may not use external AI assistants or agentic coding tools to answer questions, generate explanations or modify the submitted work.
- **Work you cannot explain.** Any code, test, diagram or documentation you cannot explain, test or modify may receive reduced or zero individual marks (Section 3).
- **Fabricated evidence.** Back-filled commit history, invented AI-log entries, or test/evaluation results that were not actually produced.
- **Confidential data and others' work.** Never share credentials, API keys, private or institutional data with an AI tool, or commit them to GitHub. Presenting another party's work as your own remains plagiarism, and the individual reflection must be your own writing.

## 18.3 Disclosure, Process Marks and Reflection

**Disclosure.** Each student must keep an **individual AI usage log showing the date, tool and model, task or section, what the tool produced, what was changed or rejected, and how the result was verified. Include this log in that student's Individual Report section of the consolidated report.** Include one consolidated group AI usage declaration in the Group Report section, confirming that all AI use has been disclosed and that every member can explain, test and modify the work submitted under their name.

**Process marks.** As AI use is permitted at Level 4, 30 marks assess the development process: 15 marks for technical understanding and ownership assessed through the viva and embedded across the individual criteria, 5 core marks for Testing, CI and Git Workflow, and 10 marks for Documentation and Deployment. These marks are assessed through the rubric in Section 16.1. **This meets the CLEAR minimum of thirty percent process marks for a Level 4 assessment.**

**Reflection (marked).** **Each student must include an approximately one-page individual reflection in their Individual Report section of the consolidated report.** It is marked under Documentation and Deployment and may be discussed during the viva. Address the following:

- Which AI tools, if any, were used, and at which stages?
- What did the AI tools do well, and what did they get wrong?
- What did you change, add or reject from the AI output, and why?
- What did you learn about your own skills and understanding?

The AI rules above were agreed with the SE3090 cohort and are published on Course Web (SE3090 → Assignments → Assignment 1). A reflection that is AI-generated, or that does not match the student's Git history and AI usage log, will not receive credit.

## 19. Academic Integrity

AI use that complies with Section 18 is permitted and expected. The following requirements apply to all submitted work, whether or not AI tools were used.

- Do not submit code or features that you cannot explain, test or modify.
- Acknowledge external libraries, APIs, tutorials, sample code and AI assistance.
- Maintain the AI usage log and submit the AI usage declaration.
- Do not expose credentials, private data or protected institutional information.
- Copying another group's code, agent design or reports, or submitting commissioned work, is a serious academic offence.
- Plagiarism in any form is not permitted, and standard SLIIT academic-integrity and plagiarism procedures apply to all submitted work.

## 20. Final Student Checklist

|                                                                                                      |                                                                                                                                             |
|------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> Required number of primary business components completed (one per student)  | <input type="checkbox"/> ASP.NET Core API and PostgreSQL working                                                                            |
| <input type="checkbox"/> JWT authentication and role-based authorization completed                   | <input type="checkbox"/> React and Flutter applications working through the shared API                                                      |
| <input type="checkbox"/> At least four specialized agents with controlled tools and structured state | <input type="checkbox"/> Validation, observability and human approval implemented                                                           |
| <input type="checkbox"/> Meaningful third-party integration completed                                | <input type="checkbox"/> Traditional testing, Agentic AI evaluation and performance testing completed                                       |
| <input type="checkbox"/> GitHub Actions CI workflow building and running tests.                      | <input type="checkbox"/> ADR completed with justified framework and architecture decisions                                                  |
| <input type="checkbox"/> React, ASP.NET Core and PostgreSQL deployed; Flutter APK generated          | <input type="checkbox"/> One consolidated report containing the Group Report, all Individual Reports, diagrams and required links completed |
| <input type="checkbox"/> Git contribution visible for every member                                   | <input type="checkbox"/> AI usage declared and no secrets committed to GitHub                                                               |
| <input type="checkbox"/> Demonstration and viva prepared with no external AI use                     | <input type="checkbox"/> Contribution statements, AI logs, group declaration and individual reflections included in the consolidated report |